

RTOS Implementation: XCMG XDE320 Plant Simulator on Zephyr / STM32H753ZI

Mario Tilocca

April 2026 – Version 1.0

Contents

1	Introduction	3
1.1	Goals of the RTOS Port	3
1.2	Platform vs. Host Differences	3
2	Hardware Overview	4
3	Thread Architecture	4
3.1	Thread Inventory	4
3.2	Thread Interaction Diagram	5
4	Plant Loop Timing	5
4.1	Timer-Semaphore Pattern	5
4.2	Timing Budget	6
4.3	Timing Flow	6
5	CAN Architecture	7
5.1	CAN RX Path	7
5.2	CAN TX Path	7
5.3	CAN Watchdog / Safe-Mode	8
6	Physics on the MCU	8
6.1	Subsystem Execution Order	8
6.2	XCMG XDE320 Parameters	9
6.3	Braking Performance	9
7	Shared State and Concurrency	10
7.1	Global State	10
7.2	Access Pattern	10
8	HTTP Dashboard	10
8.1	Module Structure	10
8.2	CSS Embedding	10
8.3	Request Flow	11
8.4	Dashboard Cards	11
9	Shell Commands	11

10 Memory Layout	12
10.1 Measured Flash Footprint	12
10.2 Measured RAM Footprint	12
10.3 Thread Stack Allocation (Actual)	13
10.4 Audit Commands	13
11 Build System	13
11.1 Dual Build Paths	13
11.2 Build Commands	14
12 Verification	14
13 Conclusion	15

1 Introduction

This document describes the embedded real-time implementation of the XCMG XDE320 electric haul-truck plant simulator on a **ST Nucleo-H753ZI** board (STM32H753ZI, ARM Cortex-M7 @ 480 MHz) running **Zephyr RTOS v3.7.0**.

The host Linux version runs the same `src/plant/` C++ source in a single-process for-loop, producing CAN frames over SocketCAN. The Zephyr port replaces only the *platform layer* (timing, threading, CAN I/O, logging, network), leaving the physics subsystems — Dugoff tyre model, 3-DOF vehicle dynamics, load transfer, wheel rotational dynamics, battery model — entirely unchanged.

1.1 Goals of the RTOS Port

1. Run the 10 ms physics step deterministically on bare metal with a hardware timer, not a software sleep.
2. Receive `ACTUATOR_CMD_1` frames via FDCAN1 (J1939 extended ID `0x98EFF021`), decode them in a dedicated CAN RX thread, and expose the command to the plant thread through a mutex-protected global.
3. Broadcast all 15 sensor TX frames (IMU, GNSS, wheel speeds, battery, vehicle state) over FDCAN1 at the rates defined in the DBC.
4. Provide a live web dashboard on Ethernet (`192.168.1.80:80`) for real-time monitoring and manual command injection.
5. Provide an interactive UART shell for diagnostics, parameter tuning, and direct plant injection without a CAN controller.

1.2 Platform vs. Host Differences

Concern	Linux Host	Zephyr Embedded
Timing	<code>std::chrono::steady_clock</code>	<code>k_uptime_get_32()</code> / <code>k_cycle_get_32()</code>
Loop control	<code>sleep_for(10ms)</code>	<code>k_timer</code> + <code>k_sem</code>
Threading	<code>std::thread</code>	<code>K_THREAD_DEFINE</code>
Mutual exclusion	<code>std::mutex</code>	<code>K_MUTEX_DEFINE</code> / <code>k_mutex_lock</code>
CAN I/O	<code>SocketCAN write(sock, ...)</code>	<code>can_send(dev, &frame, ...)</code>
CAN RX	<code>read(sock)</code> blocking	<code>can_add_rx_filter_msgq</code> + <code>k_msgq_get</code>
Logging	<code>printf</code>	<code>LOG_INF</code> / <code>LOG_WRN</code> (Zephyr backend)
Networking	N/A	Zephyr networking stack, <code>zsock.*</code>
CAN map	DBC file read at startup	<code>constexpr</code> array compiled into flash

2 Hardware Overview

Property	Value
Board	ST Nucleo-H753ZI (Nucleo-144)
MCU	STM32H753ZI
Core	ARM Cortex-M7 @ 480 MHz
Flash	2 MB
RAM	1 MB (DTCM + ITCM + AXI SRAM)
CAN peripheral	2× FDCAN (ISO 11898-1:2015)
FDCAN1 pins	PD0 (RX), PD1 (TX) — CN11 morpho pins 57/55
Ethernet	10/100 MAC + external PHY (RJ45 on board)
Debug UART	USART3 → ST-Link virtual COM (USB, no extra cable)
IP address	192.168.1.80
MAC address	02:00:5E:00:53:01 (locally administered)
Zephyr target	nucleo_h753zi

An external 3.3 V-compatible CAN transceiver (e.g. TCAN1042 or SN65HVD230) is required between PD0/PD1 and the bus differential pair.

3 Thread Architecture

3.1 Thread Inventory

Seven application threads run concurrently on a single Cortex-M7 core. Zephyr uses a fixed-priority pre-emptive scheduler; lower numeric priority preempts higher numbers.

Thread	Prio	Stack	Source file	Role
<code>stm_eth</code>	−14	1536 B	HAL driver	STM32 Ethernet DMA
<code>tcp_work</code>	−14	1024 B	Zephyr net	TCP work queue
<code>rx_q[0]</code>	−1	1536 B	Zephyr net	Network RX queue
<code>sysworkq</code>	−1	1024 B	Zephyr kernel	System work queue
<code>main</code>	0	4096 B	<code>main.cpp</code>	Boot + idle sleep
<code>can_rx_tid</code>	3	1024 B	<code>can/can_rx.cpp</code>	CAN RX + decoder
<code>plant_tid</code>	5	16 384 B	<code>plant/plant_thread.cpp</code>	Physics loop
<code>http_tid</code>	10	8192 B	<code>http/http_server.cpp</code>	Web dashboard
<code>led_tid</code>	12	512 B	<code>led/led_task.cpp</code>	Status LEDs
<code>shell_uart</code>	14	4096 B	Zephyr shell	UART shell
<code>logging</code>	14	1024 B	Zephyr LOG	Log backend
<code>idle</code>	15	320 B	Zephyr kernel	CPU idle

3.2 Thread Interaction Diagram

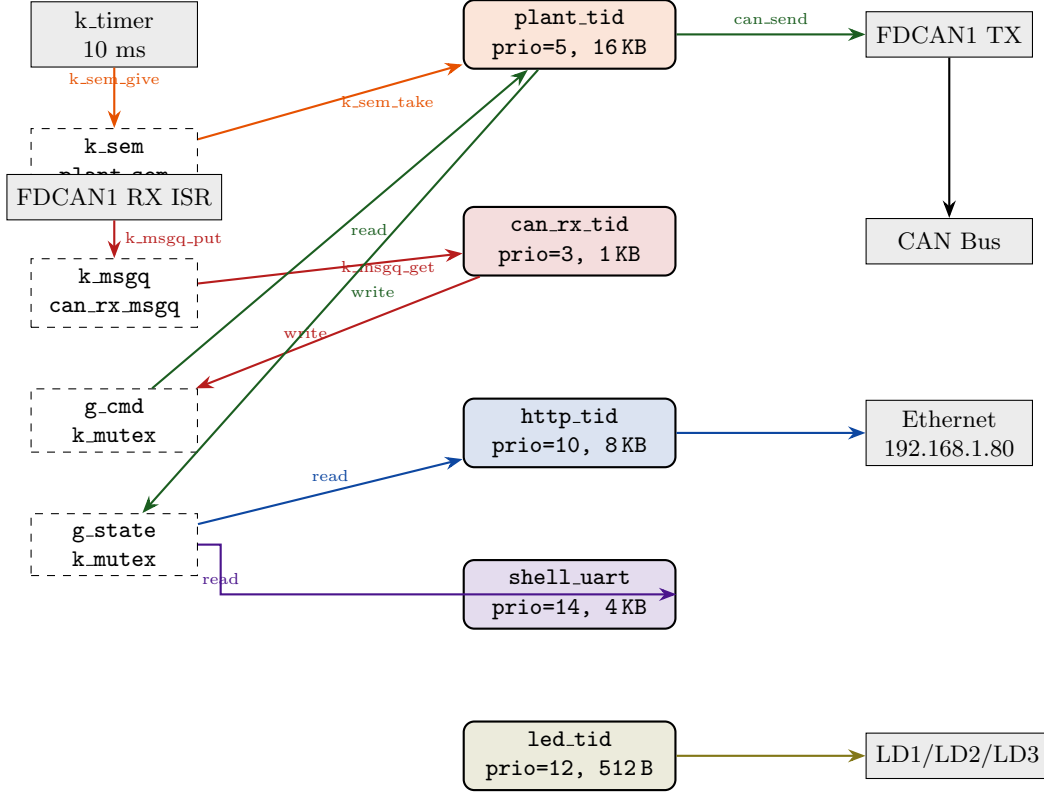


Figure 1: Inter-thread communication in the Zephyr port. The plant thread sits at the centre, consuming commands from the CAN RX thread and publishing state to the HTTP and shell threads.

4 Plant Loop Timing

4.1 Timer-Semaphore Pattern

The plant loop does *not* use `k_msleep`. Instead it uses a **hardware timer** (`k_timer`) that fires every 10 ms and gives a binary semaphore. The plant thread blocks on `k_sem_take` until the next tick arrives. This pattern ensures that the computation *budget* for one step is the full 10 ms interval and that any overrun is detected via a cycle counter.

```

1 K_SEM_DEFINE(plant_sem, 0, 1);
2 static void plant_timer_expiry(struct k_timer*) { k_sem_give(&plant_sem); }
3 K_TIMER_DEFINE(plant_timer, plant_timer_expiry, NULL);
4
5 static void plant_thread(void*, void*, void*)
6 {
7     k_timer_start(&plant_timer, K_MSEC(10), K_MSEC(10));
8     while (true) {
9         k_sem_take(&plant_sem, K_FOREVER);           // blocks until next tick
10        const uint32_t t0 = k_cycle_get_32();         // start of budget
11        const double   t_s = k_uptime_get_32() / 1000.0;
12
13        // --- physics step (see Section 5) ---
14        s_plant.step(local_state, cmd, DT_S);
15
16        // --- CAN TX ---
17        can_tx_send_all(local_state, t_s, loop_us);
18    }

```

```

19     const uint32_t dt = k_cycle_get_32() - t0;
20     const uint32_t loop_us = k_cyc_to_us_ceil32(dt); // budget check
21 }
22 }
23 K_THREAD_DEFINE(plant_tid, 16384, plant_thread, NULL, NULL, NULL, 5, 0, 0);

```

Listing 1: Timer-semaphore plant loop skeleton

4.2 Timing Budget

Activity	Typical cost	Notes
Mutex lock/unlock ($\times 4$)	$< 1 \mu\text{s}$	<code>g_cmd</code> , <code>g_state</code>
Physics step (<code>s_plant.step</code>)	200–400 μs	Dugoff + 3-DOF + battery
CAN TX (15 frames)	150–250 μs	<code>can_send</code> non-blocking
Brake diagnostics log	$< 10 \mu\text{s}$	Only when <code>brake_cmd_pct > 1</code>
Total	$< 700 \mu\text{s}$	Budget: 10 000 μs

The 10 ms budget is comfortably satisfied. The remaining 9.3 ms is spent in `k_sem_take` waiting for the next tick, during which the Cortex-M7 is idle and lower-priority threads (HTTP, LED, shell) run.

4.3 Timing Flow

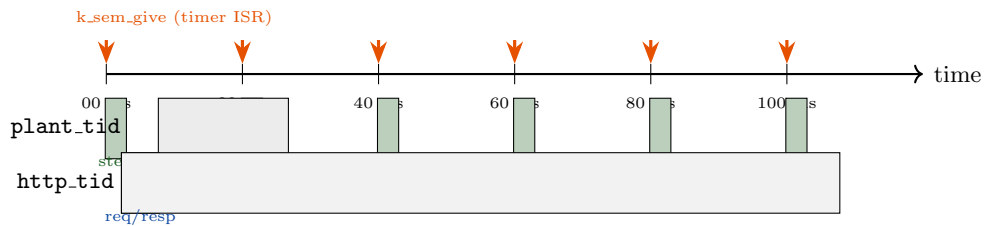


Figure 2: Simplified timing diagram showing the 10 ms plant step and the longer HTTP request/response event. The Cortex-M7 is idle between steps.

5 CAN Architecture

5.1 CAN RX Path

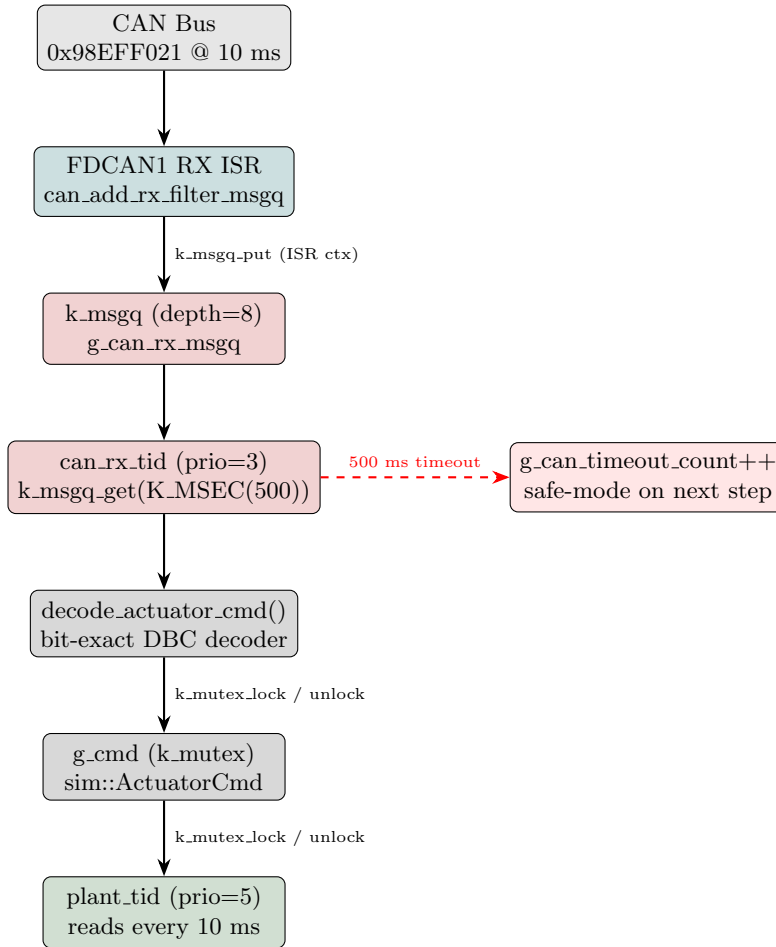


Figure 3: CAN RX data path from bus to plant thread. The ISR deposits frames into a message queue; `can_rx_tid` decodes them. A 500 ms timeout triggers safe-mode in the plant thread.

The inline decoder reads directly from the 8-byte `zf.data[]` array, using the bit positions from the DBC:

```

1 // Intel byte order (LSB-first), positions from can_map.dbc:
2 c.system_enable      = (d[0] >> 0) & 0x01;
3 c.gear_position      = static_cast<GearPosition>((d[0] >> 1) & 0x03);
4 c.steer_cmd_deg      = (int16_t)(d[1] | (d[2] << 8)) * 0.1;
5 c.drive_torque_cmd_nm = (int16_t)(d[3] | (d[4] << 8)) * 10.0;
6 c.brake_cmd_pct      = d[5] * 1.0;

```

Listing 2: ACTUATOR_CMD_1 inline decoder

5.2 CAN TX Path

`can_tx.cpp` is called once per 10 ms plant step. It packs all 15 TX frames from the `PlantState` fields and calls `can_send()` for each frame that is due (modulo the frame's `cycle_ms` period):

```

1 void can_tx_send_all(const plant::PlantState& s, double t_s, uint32_t loop_us)
2 {
3     for (uint8_t i = 0; i < can_static::k_tx_frame_count; ++i) {
4         const auto& fd = can_static::k_frames[i];

```

```

5         if (!fd.is_rx && should_send(i, t_s)) {
6             can_pack_and_send(fd, s, t_s, loop_us);
7         }
8     }
9 }

```

Listing 3: CAN TX dispatch loop (simplified)

Frame	29-bit ID	Signals	Rate
ACTUATOR_CMD_1	0x98EFF021	6	RX 10 ms
IMU_ACC	0x8CFF0028	4	TX 5 ms
VEHICLE_STATE_1	0x98FF50F0	4	TX 10 ms
WHEEL_SPEED	0x98FF3040	4	TX 10 ms
GNSS_LL	0x98FF1029	2	TX 100 ms
...	...	...	...

5.3 CAN Watchdog / Safe-Mode

The plant thread compares the current uptime against `g_last_rx_t` (updated by `can_rx_tid` on every decoded frame). If the gap exceeds 500 ms, the command is replaced by a zeroed `ActuatorCmd`:

```

1 if ((t_s - g_last_rx_t) > 0.5) {
2     cmd = sim::ActuatorCmd{}; // zero torque, zero brake, neutral gear
3 }

```

Listing 4: Watchdog in plant thread

This prevents the vehicle from running away when the CAN controller disconnects. `g_can_timeout_count` is displayed on the web dashboard under *CAN Stats* and accessible via the `can stats` shell command.

6 Physics on the MCU

6.1 Subsystem Execution Order

The `PlantModel::step()` call invokes five subsystems via the `SubsystemManager` visitor in fixed priority order:

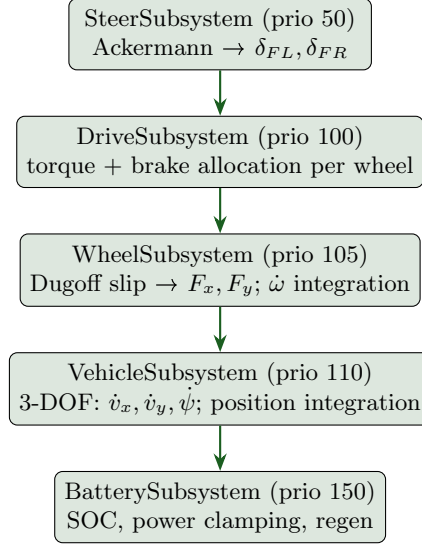


Figure 4: Subsystem execution order within one 10 ms step. Each subsystem reads from and writes to the shared `PlantState` struct.

6.2 XCMG XDE320 Parameters

Parameter	Value	Notes
Mass	218 000 kg	218 t (payload loaded)
Wheel radius	1.93 m	
Motor torque max	145 000 N m	
Motor power max	2013 kW	
Gear ratio	28.0	motor to wheel
Drivetrain efficiency	0.92	
Brake torque max	2.5×10^6 N m	2.5 MNm $\rightarrow \approx 0.6 g$
Aerodynamic drag c_d	1.85	
Rolling resistance F_{roll}	9500 N	
Max speed	17.78 m s^{-1}	64 km/h
Dugoff C_x base	280 000 N	load-scaled: $(F_z/F_{z,\text{ref}})^{0.5}$
Dugoff C_y base	220 000 N	
Reference normal load $F_{z,\text{ref}}$	800 000 N	
Peak friction μ (default)	0.72	compact gravel

6.3 Braking Performance

At full brake (`brake_cmd_pct` = 100), the torque budget per wheel is:

$$\tau_{\text{brake, wheel}} = \frac{\tau_{\text{max}}}{4} = \frac{2.5 \times 10^6}{4} = 625\,000 \text{ N m} \quad (1)$$

The equilibrium wheel-level braking force (Dugoff slip-limited) is:

$$F_{x,\text{brk}} = \frac{\tau_{\text{brake, wheel}}}{R} = \frac{625\,000}{1.93} \approx 323\,800 \text{ N per wheel} \quad (2)$$

Total deceleration:

$$a_{\text{brk}} = \frac{4 \times F_{x,\text{brk}}}{m} = \frac{4 \times 323\,800}{218\,000} \approx 5.94 \text{ m s}^{-2} \approx 0.61 g \quad (3)$$

This is below the tyre friction ceiling ($\mu F_z = 0.72 \times 535\,000\text{ N} \approx 385\,000\text{ N}$ per wheel), so the wheels do not lock. The brake diagnostics log confirms $a \approx -5.9\text{ m/s}^2$ during full braking.

7 Shared State and Concurrency

7.1 Global State

All inter-thread data lives in a small set of globals defined in `main.cpp`:

```

1 plant::PlantState g_state{}; // guarded by g_state_mutex
2 sim::ActuatorCmd g_cmd{}; // guarded by g_cmd_mutex
3 K_MUTEX_DEFINE(g_state_mutex);
4 K_MUTEX_DEFINE(g_cmd_mutex);
5
6 volatile uint32_t g_can_tx_count = 0;
7 volatile uint32_t g_can_rx_count = 0;
8 volatile uint32_t g_can_timeout_count = 0;
9 volatile double g_last_rx_t = 0.0;
10 double g_surface_mu = 0.72;

```

Listing 5: Shared globals in `main.cpp`

7.2 Access Pattern

Global	Writers	Readers
<code>g_state</code>	<code>plant_tid</code>	<code>http_tid</code> , <code>shell_uart</code>
<code>g_cmd</code>	<code>can_rx_tid</code>	<code>plant_tid</code>
<code>g_last_rx_t</code>	<code>can_rx_tid</code> , <code>http_tid</code>	<code>plant_tid</code>
<code>g_surface_mu</code>	<code>shell_uart</code>	<code>plant_tid</code>
<code>g_can_tx_count</code>	<code>plant_tid</code>	<code>http_tid</code> , <code>shell_uart</code>
<code>g_can_rx_count</code>	<code>can_rx_tid</code>	<code>http_tid</code> , <code>shell_uart</code>

Critical sections are kept short: the mutex is locked only for the duration of a struct copy (`g_state = local_state`), never during the physics computation itself.

8 HTTP Dashboard

8.1 Module Structure

The HTTP module is split across six files for maintainability:

File	Responsibility
<code>http_server.cpp</code>	Socket lifecycle: <code>socket</code> → <code>bind</code> → <code>listen</code> → <code>accept</code> → <code>close</code>
<code>http_cmd.cpp</code>	Request-line parse, query-string decode, <code>apply_web_cmd()</code>
<code>http_page.cpp</code>	HTML page builder — all <code>snprintf</code> / <code>send_str</code> calls
<code>http_html.hpp</code>	Static HTML fragments as <code>const char[]</code> constants
<code>dashboard.css</code>	All CSS rules as a real <code>.css</code> file (no C escaping)
<code>http_internal.hpp</code>	Shared internal declarations across the three <code>.cpp</code> files

8.2 CSS Embedding

`dashboard.css` is a plain CSS file editable with any editor or IDE with CSS support. It is converted to a C byte array at build time by Zephyr's `generate_inc_file_for_target()` CMake function:

```

1 generate_inc_file_for_target(app
2     ${CMAKE_CURRENT_SOURCE_DIR}/src/http/dashboard.css
3     ${CMAKE_CURRENT_BINARY_DIR}/dashboard.css.inc
4 )

```

Listing 6: CMakeLists.txt — CSS embedding

```

1 static const char kDashboardCss[] = {
2 #include "dashboard.css.inc"    // comma-separated hex bytes from file2hex.py
3     '\0'
4 };

```

Listing 7: http_page.cpp — including the generated array

Incremental builds detect changes to `dashboard.css` and regenerate the `.inc` file automatically; only `http_page.cpp` needs recompilation.

8.3 Request Flow

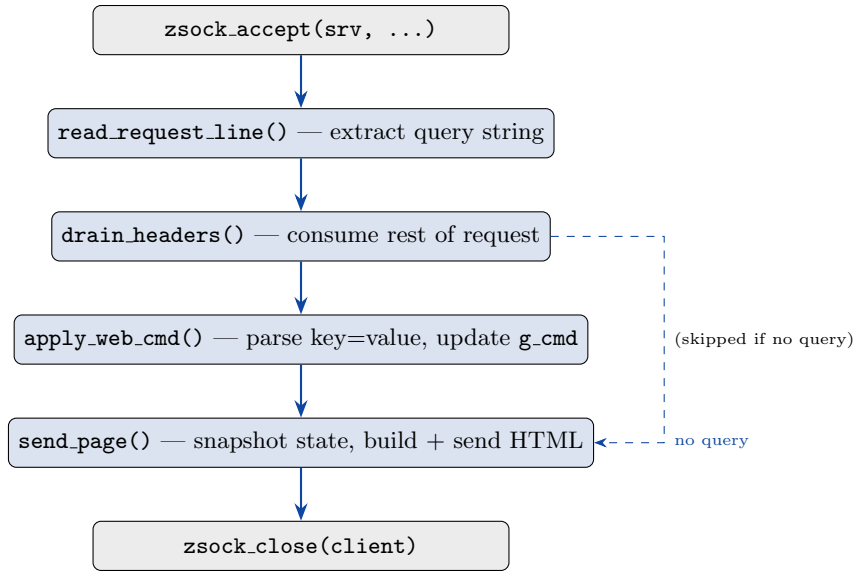


Figure 5: HTTP server request lifecycle for one browser connection.

8.4 Dashboard Cards

1. **Plant State** — v_x , v_y , yaw, yaw rate, position, steer angle, SOC, per-wheel ω , surface μ
2. **Actuator Command** — enable, gear, torque, brake, steer (current command sent to plant)
3. **CAN Stats** — TX frames, RX frames, timeout count, last RX timestamp
4. **Controls** — quick-action buttons (STOP, Drive FWD, Drive REV, steer $\pm 5^\circ$) and a manual-inject form; all via HTTP GET
5. **Kernel Threads** — per-thread name, priority, stack used/total (via `k_thread_foreach`)
6. **Vehicle Info** — hardcoded XCMG XDE320 specification table

9 Shell Commands

All commands are implemented in `shell/debug_cmds.cpp` using Zephyr's `SHELL_CMD_REGISTER` macro.

Command	Action
plant state	Dump all PlantState fields
plant inject <steer> <torque> <brake>	Inject actuator command directly
plant mu <val>	Set surface friction coefficient (0.1–1.0)
plant reset	Zero all plant state
can stats	TX/RX frame counts, timeouts, last RX timestamp
can rx_frame	Last decoded ACTUATOR_CMD_1 fields
can tx_test <steer> <torque> <brake>	Encode + transmit test frame (loopback)
can map	Print static CAN frame table (16 entries)
vehicle info	XCMG XDE320 parameters + live surface μ
network mac	MAC and IP from <code>net_if_get_default()</code>
system uptime	HH:MM:SS and ms

10 Memory Layout

10.1 Measured Flash Footprint

Measured with `west build -t rom_report` on a full Phase 4 build. Total Flash: 236 957 B $\approx$ 231.4 KB — **11.3 %** of the 2 MB Flash budget; 1.77 MB free.

Region	Budget	Measured	Notes
Total Flash	2 MB	231.4 KB	11.3 % utilised
App (zephyr/src/)	—	12.7 KB	threads, HTTP, CAN, shell
can/can_tx.cpp	—	3416 B	<code>can_tx_send_all()</code>
shell/debug_cmds.cpp	—	2372 B	11 command handlers
plant/plant_thread.cpp	—	1488 B	<code>plant_thread()</code>
http/http_page.cpp	—	1642 B	<code>send_page()</code>
kDashboardCss (rodata)	—	1758 B	embedded CSS
Shared plant physics	—	12.8 KB	subsystems from <code>src/plant/</code>
Zephyr RTOS	—	121.1 KB	kernel + subsys + drivers
Newlib / unresolved	—	61.5 KB	math, C stdlib

10.2 Measured RAM Footprint

Measured with `west build -t ram_report` on a full Phase 4 build targeting `nucleo_h753zi`. Total static RAM: 241 075 B $\approx$ 235 KB — **22.9 %** of the 1 MB SRAM budget; 765 KB free.

Region	Budget	Measured	Dominant item
Total SRAM	1 MB	235 KB (241 075 B)	22.9 % utilised
App code	—	27.4 KB (28 100 B)	stacks + shared state
plant_thread.cpp	—	16 760 B	stack 16 448 B
http_server.cpp	—	8488 B	stack 8 256 B
can_rx.cpp	—	1372 B	stack 1 088 B + msgq
led_task.cpp	—	808 B	stack 576 B
main.cpp	—	672 B	<code>g_state</code> 560 B
Zephyr kernel	—	133.6 KB	<code>kheap</code> 128 KB
Networking subsystem	—	34.0 KB	TCP slab, pkt/buf pools
ETH driver	—	14.5 KB	DMA RX+TX buffers*
Logging subsystem	—	5.5 KB	ring buffer + thread

* ETH DMA buffers reside in a dedicated `eth_stm32` SRAM region at `0x30040000`, separate from the main AXI SRAM.

Heap: `kheap__system_heap` = 128 KB (configured pool). Runtime peak is inspectable via the Zephyr shell:

```
1 uart:~$ kernel heap
```

If RAM becomes constrained, reduce `CONFIG_HEAP_MEM_POOL_SIZE` in `prj.conf`; 64 KB is sufficient for current allocations.

10.3 Thread Stack Allocation (Actual)

Zephyr adds alignment padding to each `K.THREAD_DEFINE` stack; the table below shows the aligned sizes from `ram_report`.

Thread	Configured	Actual	Base address
<code>plant_tid</code>	16 384 B	16 448 B	0x24008080
<code>http_tid</code>	8192 B	8256 B	0x24005c00
<code>z_main_stack</code>	4096 B	4160 B	0x2400ecc0
<code>z_interrupt_stacks</code>	2048 B	2112 B	0x2400e300
<code>can_rx_tid</code>	1024 B	1088 B	0x24007c40
<code>sys_work_q</code>	1024 B	1088 B	0x2400fd00
<code>logging</code>	1024 B	1088 B	0x2400c0c0
<code>led_tid</code>	512 B	576 B	0x240059c0
<code>z_idle_stacks</code>	320 B	384 B	0x2400eb40
Total	—	35 200 B ($\approx$ 34.4 KB)	

10.4 Audit Commands

```
1 west build -t ram_report # per-object static RAM
2 west build -t rom_report # per-object Flash (.text + .rodata)
3 uart:~$ kernel heap     # runtime heap peak (Zephyr shell)
```

Listing 8: RAM and Flash audit

11 Build System

11.1 Dual Build Paths

Both the host simulator and the embedded firmware compile from the same `src/` and `utils/` source tree. Only `CMakeLists.txt` and the platform source files differ.

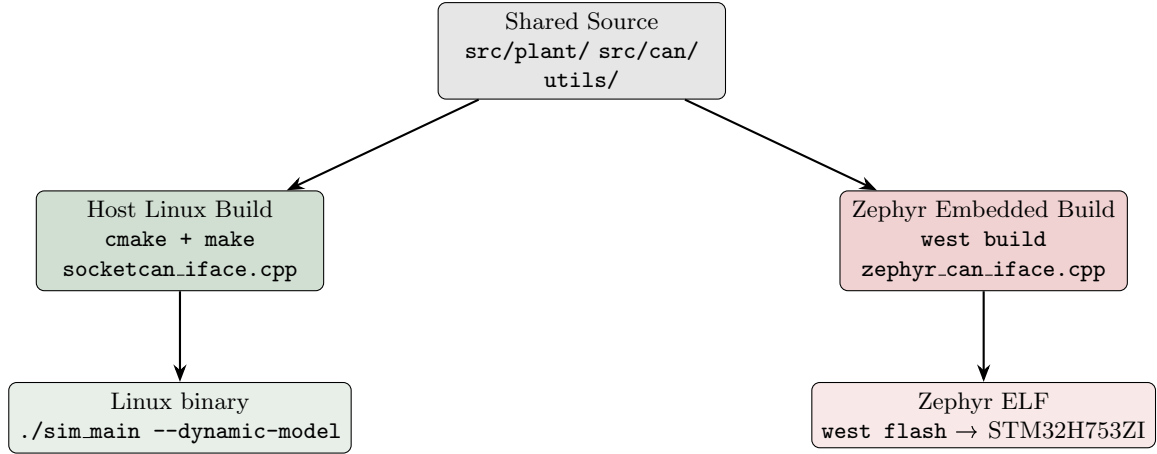


Figure 6: Dual build paths sharing the same plant and CAN source.

11.2 Build Commands

```

1 cd ~/zephyrproject
2 rm -rf build # required when prj.conf or overlay changes
3 west build -b nucleo_h753zi \
4   ~/repos/vehicle-Dynamics-Sim-Can/zephyr
5 west flash
6 picocom -b 115200 /dev/ttyACM0 # UART shell monitor

```

Listing 9: Complete clean build and flash sequence

Dashboard is available at <http://192.168.1.80> once the Ethernet link is up (typically within 2–3 s of boot).

12 Verification

1. Boot banner — UART shows:

```

*** Booting Zephyr OS build v3.7.0 ***
[INF] xcmg_sim: XCMG XDE320 plant thread started (dt=10 ms, prio=5)

```

2. **Plant state** — shell command `plant state` shows live v_x , yaw, SOC, wheel speeds; values change after `plant inject`.
3. **CAN RX** — connect a CAN controller, send `ACTUATOR_CMD_1` at 10 ms; `can rx_frame` shows decoded fields; `can stats` shows incrementing RX count.
4. **CAN TX** — CAN analyzer on bus shows 15 TX frames at the rates in the DBC (5 ms IMU, 10 ms VEHICLE_STATE, 100 ms GNSS, ...).
5. **Web dashboard** — browser at <http://192.168.1.80>:
 - *Drive FWD* button → v_x rises toward 17.78 m s^{-1}
 - *STOP* button → $a_{\text{long}} \approx -5.9 \text{ m/s}^2$ ($\approx 0.6 g$)
 - Kernel Threads card shows all thread names and stack usage
6. **Brake diagnostics** — UART log during hard braking:

```
[brk] t=12.34 vx=8.320 a=-5.91 Fx=-1291kN tau=-2500kNm mode=dyn  
[brk] omega FL=4.31 FR=4.31 RL=4.31 RR=4.31 ref=4.31 rad/s
```

7. **Safe-mode** — disconnect the CAN controller; within 500 ms the *CAN Stats* timeout counter increments and torque drops to zero.

13 Conclusion

The Zephyr port demonstrates that a high-fidelity vehicle dynamics model (3-DOF rigid body, Dugoff tyre, per-wheel rotational dynamics, Ackermann steering, battery model) can execute deterministically on a mid-range Cortex-M7 microcontroller with a 10 ms timestep, leaving over 93% of the 10 ms budget unused.

The dual build architecture — shared `src/plant/` compiled for both Linux and Zephyr — minimises code divergence and allows the same test vectors to be applied to both the host simulation and the embedded target. The web dashboard, UART shell, and brake diagnostics provide sufficient observability for iterative tuning without a full CAN analysis setup.